Implementation Notes

What this document is: a complete record of what was built, why each decision was made, and what each change was worth — written for someone with **no background in search or recommender systems**.

Every technical term is explained in plain language, with a real example from this catalog, before it is used. If you hit a word you don't recognise, it is either defined in [§2 Vocabulary](#2-vocabulary) or in the [Glossary](#glossary) at the end.

Each stage section ends with **"Ideas for this stage"** — enhancements specific to that piece of the pipeline. Enhancements that cut across several stages, or that are about how the stages are *orchestrated*, are collected in [§10](#10-cross-cutting-and-orchestration-ideas) at the end.

Result: the supplied starter agent scores **0.1067**. This implementation scores **0.8592** — roughly 8x — using no AI model, no network connection, and nothing outside the Python standard library.

Table of contents

---

1. What the agent has to do

There is a **catalog** of 50,000 clothing, shoe and jewellery products (`data/catalog.jsonl`). One of them is the **target** — a specific item a specific customer actually bought. The agent is never told which one.

A **session** is one conversation. It runs for at most **10 turns**. On each turn:

- the customer sends a message,
- the agent replies with a sentence, optionally *a question*, and optionally *a list of up to 10

product IDs*.

The session ends the moment the target appears anywhere in that list — or after turn 10, which counts as a failure.

Here is a real session from the public set, produced by the finished agent:

```
session public_0008
TARGET: Hanes Womens Wireless Bra, Full-Coverage Pullover Stretch-Knit Bra

turn 1  customer> I'm looking for Bras Everyday Bras. A key requirement is: nylon.
        agent   > To narrow this down: is there anything else that matters
                  for this one?                     [asks: other]  no list yet

turn 2  customer> For that, what matters is: 96% Nylon, 4% Spandex; Pull-On closure.
        agent   > (same question)                   [asks: other]  no list yet

turn 3  customer> For that, what matters is: Hand Wash Only.
        agent   > (same question)                   [asks: other]  10 products
                  -> TARGET at position 1.  HIT on turn 3.
```

(That is a real run, reproducible with the agent as it stands. Not every session looks like this — in some, the agent becomes confident enough to show a list on turn 2; §S7 explains when and why.)

Two things in that transcript are the whole story of this project, and both are explained in detail later:

1. The customer **only volunteered information because the agent asked a question**. Had the

agent not asked, turns 2 and 3 would have contained nothing useful.

2. The agent **deliberately showed no products on turns 1 and 2**, even though it could have.

§3 explains why holding back is worth more than it costs.

---

2. Vocabulary

Read this once; everything later depends on it.

The data

Term	Meaning
Catalog	The 50,000 products. One JSON object per line in `data/catalog.jsonl`.
`parent_asin`	Amazon's product ID, e.g. `B07KCFS4VC`. The *only* thing that is scored — a recommendation is right only if this string matches the target exactly. Close is not counted.
Product fields	`title`, `features` (bullet points), `description`, `price`, `categories`, `details` (a dictionary like `{"Department": "Womens"}`), `average_rating`, `rating_number`, `store` (the brand).
Session	One conversation with one customer, targeting one product.
Turn	One customer message plus one agent reply. Maximum 10 per session.
Target	The hidden product the agent must find.
Scenario	Sessions come in four flavours: **buying** (customer states a firm requirement immediately), **browsing** (customer starts vague), **intent override** (customer changes their mind partway through), **boundary** (customer has no opinion on something the agent asks about).

The two halves of any search system

This distinction is the single most important idea in the document, and the architecture is built around it.

> **Retrieval** = *finding* a shortlist of plausible candidates out of 50,000. > **Ranking** = *ordering* that shortlist so the best one is first.

They are different jobs with opposite requirements:

- Retrieval must be **generous**. If the target isn't in the shortlist, nothing downstream can

save you. Retrieval is allowed to include junk.

- Ranking must be **strict**. Its job is to separate the one right answer from 299 near-misses.

A technique can be excellent at one and terrible at the other. §6 documents a case where exactly that happened, and where moving one technique from retrieval to ranking turned a large loss into a large gain.

How text search works (BM25)

The agent finds products by matching words. The scoring function used is **BM25**, a standard text-search formula. You don't need the formula — you need three intuitions:

1. **Rare words count more than common words.** If a customer says "kandinsky", that word appears

in maybe one product in the whole catalog, so it is enormously informative. If they say "imported", which appears in tens of thousands, it tells you almost nothing. BM25 weighs each word by its rarity across the catalog. (The technical name for this rarity weighting is **IDF**, *inverse document frequency*.)

2. **More matches are better, but with diminishing returns.** A product mentioning "leather" five

times isn't five times more relevant than one mentioning it once.

3. **Which field matched matters.** A word in the product's `title` is stronger evidence than the

same word buried in a long `description`. This is configurable — see `DEFAULT_WEIGHTS` at `src/index.py:25`.

The search index itself is **SQLite FTS5**, a full-text search engine built into Python's standard library. That is why this project needs no third-party packages: the search engine is already on your machine.

Measuring success

Term	Meaning
Top-k	The first *k* items of a ranked list. Here *k* = 10.
Hit	The target appeared in a list the agent showed.
Hit Rate@10	Fraction of sessions that ended in a hit. `0.94` = the agent found the target in 94% of sessions.
Rank	The target's position in the list. Position 1 is best.
Reciprocal rank (RR)	`1 ÷ rank`. Rank 1 → `1.0`. Rank 2 → `0.5`. Rank 10 → `0.1`. A miss → `0`. This rewards being *first* far more than merely being *present*.
MRR	Mean Reciprocal Rank — the average RR across all sessions. `0.79` roughly means "on average the target lands around position 1.3".
MTTC	Mean Turns To Conversion — the average turn number on which the hit happened. A miss counts as turn 11. Lower is better.

Terms specific to this project

Term	Meaning
Facet	A structured attribute of a product with a limited set of values — material (`leather`, `cotton`), colour (`black`, `navy`), price band, category. Contrast with free text. Extracted by `src/facets.py`.
Constraint span	A short phrase the customer said, normalised for matching — e.g. `"stainless steel band"`. §S6 explains why these are so powerful here.
`ask_attribute`	A structured field in the agent's reply naming *what* it is asking about (`material`, `color`, `budget`, `other`, ...). Critically, this — not the English sentence — is what the customer's simulator reads.

---

3. How the score is calculated

All of this lives in `evaluator/local_evaluator.py`, which is organizer-owned and **must never be modified**.

Step 1 — cleaning the agent's list

`normalize_recommendations` (`evaluator/local_evaluator.py:95`) silently discards:

- IDs that aren't in the catalog,
- duplicates,
- anything after the first 10.

"Silently" matters: a malformed recommendation isn't an error, it just quietly wastes a slot.

Step 2 — checking for a hit

```
if override_applied and target in ranked:
    best_rank = ranked.index(target) + 1
    hit_turn = turn
    break
```

— `evaluator/local_evaluator.py:252`

Note the `break`. **The session stops at the first hit.** This has a consequence that shapes the entire agent design, covered below.

Step 3 — the four metrics

From `metric_summary` (`evaluator/local_evaluator.py:188`) and line 280:

```
HitRate@10 = hits / N
MRR        = average(1 / rank),  a miss contributes 0
MTTC       = average(hit turn),  a miss counts as turn 11
Efficiency = (11 - MTTC) / 10,   clamped to the range 0..1

Score = 0.50 x HitRate@10  +  0.30 x MRR  +  0.20 x Efficiency
```

For this implementation:

```
0.50 x 0.940  +  0.30 x 0.7911  +  0.20 x 0.7595  =  0.8592
```

The five consequences that drive every design decision

1. The first list containing the target freezes both rank and turn. Because of that `break`, you never get a second chance to rank better. If you show a list on turn 2 and the target happens to be sitting at position 9, you have permanently locked in `RR = 0.11` — even though by turn 4 you might have ranked it first. This is the single most important strategic fact in the challenge.

2. Wrong recommendations are free. There is no penalty for showing 10 irrelevant products. The only cost is the turn. So the dangerous failure mode is not *being wrong* — it is *being confident too early*.

3. `ask_attribute` **is the only tap for information.** The simulated customer (`customer_reply`, `evaluator/local_evaluator.py:166`) reveals a new constraint **only** when the agent's reply sets `ask_attribute`. If it is `null`, the customer answers "ask me about one specific attribute" and discloses nothing. An agent that never asks never learns anything.

4. An exception is an invisible failure. The evaluator wraps each turn in `try / except Exception` (`evaluator/local_evaluator.py:241`) and converts a crash into an empty response. You lose the session and see no error message.

5. Override sessions cannot convert early. For intent-override scenarios, `override_applied` stays `False` until the customer's change of mind arrives on turn 3 or 4. Hits before that are ignored entirely, which puts a hard floor under that scenario's MTTC.

What each improvement is actually worth

With 200 sessions, here is the score change from improving a *single* session:

Change to one session	Score change
Turn a miss into a hit (rank 5, turn 4)	**+0.0035**

Change to one session	Score change
Move an existing hit from rank 10 to rank 1	**+0.00135**
Reach the same hit one turn sooner	**+0.0001**

Read those numbers together and they give a clear instruction:

> **Converting a miss is ~2.6x more valuable than fixing a rank, and fixing a rank is ~13x more > valuable than the turn it costs to earn.** > > Therefore: *spend turns freely to gather information; never rush a list out.*

Almost every design choice in §5 follows from this one table. It is why the agent stays silent for the first two turns.

---

4. Why the supplied baseline scored 0.1067

The original `starter/agent.py` was about 100 lines. Its `respond()` did three things: tokenise **the current message**, run one BM25 query, return the top 10. It scored:

Hit Rate@10	0.125
MRR	0.068
MTTC	9.81
Score	**0.1067**

Three flaws, in order of cost:

It never asked a question. `ask_attribute` was hard-coded to `None`. By consequence #3 above, the customer therefore never disclosed anything. The agent's entire knowledge of the customer was the opening sentence — for all ten turns.

It was stateless. Even if the customer *had* said something useful, `respond()` only ever looked at `user_message`, the current turn. Everything said earlier was discarded.

It never reranked. Whatever order BM25 returned was the order shown.

The MTTC of 9.81 tells the story: out of 10 possible turns, the average session burned 9.81 of them. The agent was effectively making the same guess ten times in a row, because nothing ever changed between turns.

The key realisation: the biggest available win was not a better search engine. It was *asking a question and remembering the answer*. Fixing only that — before touching retrieval or ranking at all — took the score from **0.1067 to 0.7811**.

---

5. Stage-by-stage changes

The agent is split into one module per job, under `src/`. `starter/agent.py` remains the file the evaluator imports; it is now a thin adapter that wires the stages together and owns the response format.

Reading order for the code: `starter/agent.py` → `src/state.py` → `src/retrieval.py` → `src/rerank.py`.

S0 — Experiment harness

File: `tools/sweep.py`

What this stage does

It isn't part of the agent. It is the tool used to *measure* the agent — run several configurations and print a comparison table.

Why it was needed

Two problems made experimentation painful.

Speed. Building the search index over 50,000 products takes about 4 seconds, and the original setup rebuilt it for every configuration tested. Comparing four ideas meant four rebuilds, and early runs timed out at two minutes.

Self-deception. There are only 200 public sessions, but **800 private sessions decide the final result**. If you tune ten parameters against 200 sessions and keep whatever scores highest, you will reliably pick settings that fit the noise in those particular 200 — and they will not transfer. This is called **overfitting**.

What changed

Shared index. The index is now cached per catalog path (`load_index` in `src/index.py`), so constructing many `Agent` objects builds it once. A full 200-session evaluation dropped from timing out to **~13 seconds**.

Dev/holdout split. `split_samples()` divides the 200 sessions into 120 **dev** and 80 **holdout**, stratified so each split keeps the same mix of scenarios. All tuning happens on dev. Holdout is looked at only to confirm a change survives on data it wasn't tuned against. Where dev and holdout disagreed, holdout won — twice, this changed what shipped (see §6).

The split is deterministic (sorted by `sample_id`), so it needs no stored file and never drifts.

```
python3 tools/sweep.py --split dev
python3 tools/sweep.py --split holdout
python3 tools/sweep.py --split all --configs floor,rerank,infogain
```

Measured effect

No direct score. Every measurement in this document exists because of it.

Ideas for this stage

- **Confidence intervals.** With 80 holdout sessions, a difference under about `0.02` is

indistinguishable from noise. The harness reports point estimates only. Bootstrap resampling over sessions would print an interval, turning "is this better?" from a judgement call into a measurement.

- **Per-session diffing.** The harness compares aggregates. A mode that lists which *specific*

sessions changed between two configurations would make regressions far faster to diagnose — currently that requires an ad-hoc script.

- **Automated parameter search.** Several constants were swept by hand

(§S7, §S4). A small random or grid search driven from the config matrix would cover more ground, though it raises overfitting risk and would need the confidence intervals above to be safe.

- **Cache the catalog parse.** `catalog_index()` re-parses 58MB of JSON per harness invocation

(~2s). A pickled cache would speed up repeated runs from the shell.

S1 — Catalog index

Files: `src/index.py`**,** `src/text.py`

What this stage does

Reads the 50,000-product catalog once and builds the structures every later stage queries: a full-text search index, and a trimmed copy of each product's data.

Why it was needed

The original code built its index inline inside the agent and kept nothing else, so no other component could ask questions like "what colour is this product?" — the information was inside the search engine and unreachable.

What changed

One pass, three products. `CatalogIndex._build()` walks the file once and produces:

1. The **FTS5 full-text index** — seven columns (`title`, `categories`, `features`, `details`, `store`, `description`), so field-specific weighting is possible.
2. **Trimmed product records** — `parent_asin`, title, categories, brand, price, ratings, and a normalised text blob. Deliberately not a second full copy of the 58MB file.
3. **Normalised text** — this one is subtle and it matters. Product text is stored with all punctuation stripped and words joined by single spaces:

`` `"Hide & Drink, 100% Leather, Buckle closure" becomes "hide drink 100 leather buckle closure"` ``

The reranker (§S6) checks whether a phrase the customer said appears inside a product's text. Without matching normalisation on both sides, `"100 leather"` would never match `"100% Leather"` and the strongest signal in the system would silently do nothing.

Shared caching. `load_index()` returns one index per catalog path, so repeated `Agent` construction is nearly free.

Word handling (`src/text.py`). Two lists of words are removed before searching:

- *Stopwords* — conversational filler: "the", "for", "looking", "preference".
- *Boilerplate* — Amazon catalog filler: "imported", "closure", "machine", "wash", "dimensions".

These appear in tens of thousands of products, so they add query cost without discriminating.

Measured effect

Index build **4.13s**, one time at startup. Peak memory **~282MB**. Per-turn latency **16.6ms mean, 19.8ms at the 95th percentile**.

Ideas for this stage

- **Tune the field weights.** `DEFAULT_WEIGHTS` at `src/index.py:25` is `(0.0, 6.0, 4.0, 2.5, 2.5, 1.5, 1.0)` — inherited verbatim from the supplied baseline and **never tuned**. Those seven numbers control how much a `title` match counts versus a `description` match. This is probably the cheapest untapped gain in the repo: the harness exists, the parameter is a single tuple, and nobody has ever checked whether the baseline's guess was any good.
- **Learn the boilerplate list instead of writing it.** ~~Compute document frequency across the catalog and drop the top ~200 terms.~~ **Done, but not the way this originally proposed — the original proposal was wrong.** Ranked by document frequency, `polyester` is 72nd, `cotton` 83rd, `black` 97th, `leather` 111th and `spandex` 141st. Dropping the top 200 would delete every one of them, and those are precisely the constraints the customer discloses, because `intent_card()` inserts a material at position 0 and a colour at position 1 of every card (§S1 explains why the customer quotes catalog copy). Meanwhile `asin` — a genuine member of the hand-written list — sits at rank 10,379 with 0.0% document frequency. Frequency fails in both directions.

  What works is *where* a token occurs, not how often. Amazon's structural metadata lives in the `details` dict, and those tokens appear almost nowhere else — `department` 100% of its occurrences, `dimensions` 99.6%, `manufacturer` 99.6%, `inches` 97.7% — while attribute values are spread across title, features and description: `spandex` 1.2%, `cotton` 2.5%, `polyester` 3.3%, `black` 13.4%. The gap between 16% and 96% is empty of real attribute words, so the threshold is read off the catalog's own distribution rather than tuned against a score. `tools/build_stoplist.py` applies that rule and writes `src/stoplist.py`; it reads `data/catalog.jsonl` and never opens `data/public_set.jsonl`, so it cannot fit the public sessions. The learned list reproduces 14 of the 24 hand-written terms and finds 22 more that nobody thought to write down — every month name and year 2014–2022, harvested from "Date First Available: August 15, 2019".

  Half the list is not learnable, and `src/text.py` now says so. `imported`, `machine`, `wash`, `closure` and `care` are care-and-origin phrases in `features`/`description`. Two statistics were tested and both fail to separate them from real attributes: by document frequency `closure` (38.6% of products) outranks `polyester` (21.8%), and by spread across the 12 largest category buckets `polyester` (CV 0.51) falls between `imported` (0.49) and `wash` (0.60), with

`black`/`white` (0.34/0.31) landing inside the scaffolding band. What separates them is that a shopper does not choose between "imported" and "not imported" — semantics, not frequency. Those ten stay hand-written, with the evidence recorded beside them so the experiment is not retried.

Measured effect: none. Public set identical to six decimal places (0.912526 both ways), adversarial set –0.0001, dev and holdout flat. Boilerplate removal strips a median of one token from a ten-token query and `MAX_QUERY_TERMS = 60` never binds — 0 of 200 sessions come close. It ships for auditability and for robustness on a catalog nobody has hand-inspected, not for points. Measurements were taken in one process with everything else held constant; see `tests/test_stoplist.py` for the invariants that keep a future regeneration honest.

- **Persist the index.** 4.1s of startup could become near-zero by writing the FTS5 index to a

file instead of `:memory:`. Only worth it if startup time is ever judged.

- ~~**Dead code.** `search_phrases()` (`src/index.py:119`) has had no caller since the phrase route

was removed in §6. It should go.~~ **Done** — deleted, along with `DialogState.query_terms()` (`src/state.py`), which also had no caller. Both deletions verified score-identical: public 0.915887, hard 0.794375, 57/57 tests.

S2 — Intent router

File: `src/router.py`

What this stage does

Classifies the opening message as **buying** (customer knows what they want) or **browsing** (customer is exploring).

Why it was needed

The problem brief calls for "dual-track routing" as a core pillar: a decided customer and an exploring one deserve different treatment.

What changed

Classification uses **linguistic cues**, not exact string matching:

```
BUYING_CUES   = "key requirement | must be | i need | it has to | ..."
BROWSING_CUES = "still exploring | just looking | not sure | open to | ..."
```

This was a deliberate choice with a trap avoided. The local evaluator builds its opening lines from fixed templates (`initial_message`, `evaluator/local_evaluator.py:154`), so matching those exact strings would score perfectly here — and could fail completely on the private sessions if the organizer paraphrases them. Cue matching degrades gracefully; template matching fails silently.

Unknown phrasing defaults to **browsing**, because mistaking a vague customer for a decided one commits to constraints they never stated, whereas the reverse costs at most one extra question.

Measured effect — and an honest scope reduction

The router was first built to change *retrieval*: a wider candidate pool for browsing (400), a narrower one for buying (200). Measured:

	dev	holdout
Router affects retrieval	0.8715	0.8373
Router does not	0.8715	0.8391

Identical on dev, 0.002 worse on holdout. The reranker (§S6) already resolves both tracks well, so there was nothing left for routing to fix.

Rather than keep a feature that does nothing and describe it as a pillar, the router was **scoped down** to the job it genuinely does: phrasing. A browsing customer hears "To point you in the right direction: …"; a buying customer hears "To narrow this down: …". That is a real product requirement even where it is not a scoring one, and the measurement

is recorded in the module docstring so the next reader doesn't repeat the experiment.

Ideas for this stage

- **Route the *question*, not the retrieval.** The measurement showed routing retrieval is

pointless, but nobody tested routing the clarification policy. A buying customer who has already stated a hard requirement might be better served by a *narrow* confirming question, while a browser needs broad ones. That is a different lever and remains untested.

- **Detect scenario, not just track.** The router distinguishes two of four scenarios. Detecting

boundary customers (people who answer "I have no preference") early would let the agent stop spending questions on someone who won't answer them — currently that is only learned after a wasted turn, via `dead_attributes` in `src/state.py`.

- **Confidence-weighted routing.** `classify()` returns a hard label. Returning a confidence, and

blending behaviour when uncertain, would avoid an all-or-nothing decision made on a single sentence.

S3 — Dialog state

File: `src/state.py`

> **This is the stage that carries the score.** Everything else in this document is a refinement > on top of it.

What this stage does

Remembers the conversation: what the customer has said, when they said it, what they have declined to answer, and whether they changed their mind.

Why it was needed

The baseline was stateless. Recall consequence #3 from §3: the customer discloses information only in response to a question, and the baseline never asked one. So the baseline was solving a much harder problem than the one posed — identifying one product in 50,000 from a single vague sentence like *"I'm looking for Watches Wrist Watches"*.

Here is what the agent actually receives across a session once it starts asking:

```
turn 1  "I'm looking for Watches Wrist Watches. Stainless Steel Band"
turn 2  "For that, what matters is: Water Resistant; 3 Year Battery."
turn 3  "For that, what matters is: Day / Date Indicator; Stainless Steel Band."
```

Every session hides **exactly four** constraints, disclosed up to two per answered question. Accumulating them across turns instead of discarding them is the difference between describing "a watch" and describing "*this* watch".

What changed

`DialogState` stores every message with **provenance** — the turn it arrived on and a weight. It exposes two views of the same history:

- `full_text()` — everything the customer has said. Maximum recall.
- `focused_text()` — only the currently authoritative turns. Maximum precision.

Until the customer changes their mind, these are identical.

Intent override — a deliberate deviation from the brief. The problem statement calls for "slot erasure and rewriting": when the customer reverses a preference, delete the old one. We **down-weight instead of erasing**, to `PRE_OVERRIDE_WEIGHT = 0.35` (`src/state.py:38`).

The reason is specific to how this evaluator constructs an override. Looking at `behavior_for()` (`evaluator/local_evaluator.py:78-86`), the "discarded" preference is *still drawn from the target product's own metadata*. Erasing it therefore throws away accurate information about the very item we are trying to find, and costs score. Down-weighting keeps the evidence usable while letting the post-override turns dominate, and `focused_text()` gives retrieval a clean post-override view in case the private simulator uses genuine decoys instead.

This deviation is flagged here, in `README.md`, and in the module docstring, because a reader comparing the code to the brief will otherwise think it is a bug.

Two other signals the state tracks, both read by the clarification policy:

- `dead_attributes` — when the customer says "I don't have a preference for material", that

attribute is recorded as dead so the agent never asks again. Re-asking a declined question wastes a turn and reads as not listening.

- `productive_turns` — whether an answer actually disclosed something new. This is how the

policy tells the difference between "the customer is still telling me things" and "I have exhausted this line of questioning".

Measured effect

0.1067 → **0.7811.** Roughly 90% of the total improvement in this project comes from this stage plus the decision to ask a question at all.

Ideas for this stage

All four override ideas below were investigated together and closed as not worth pursuing. The reason is one measured fact: `behavior_for()` draws both `old_value` and `new_value` from the *same target product's* intent card, and across all 46 override sessions in the public and hard sets, **not one replaces an exclusive facet value with a different one** — 25/30 public overrides are cross-slot (`"Buckle closure"` → `"leather"`), 4/30 are `feature` → `feature` (`"Stainless Steel Band"` → `"Water Resistant"`, both true of the target), and the single `material` → `material` case repeats the same value. The override is an *emphasis shift*, not a retraction. Full measurements in `docs/team/rerank_signals.md` §6-§8.

- ~~**Tune the override weight.**~~ **Closed.** `PRE_OVERRIDE_WEIGHT = 0.35` is inert *and* has

nothing to express: there is no discarded constraint to down-weight, so no value of the constant is more correct than another.

- ~~**Per-constraint provenance, not per-turn.**~~ **Closed.** Built as a four-variant ablation

over the conflict-scoring path (turn-1 exclusion × full-vs-focused history); every variant scored flat or worse than the shipped agent. Provenance is real — `focused_text()` in the conflict path turns out to be nothing but a turn-1 filter — but acting on it more precisely loses score, because the turn-1 category framing is a genuine constraint.

- ~~**Detect *partial* overrides.~~ Closed.** A partial override is the *only* kind this

simulator produces, and the correct handling of it is to add the new constraint without touching the old one — which is already what the agent does.

- ~~**Contradiction detection.**~~ **Closed for this evaluator.** `customer_reply()` only ever

adds constraints, all drawn from one target's card, so the customer can never state two incompatible ones. Still a real product behaviour; not a scoring lever here.

S4 — Clarification policy

File: `src/policy.py`

What this stage does

Decides **what to ask about next** — the value placed in `ask_attribute`. Legal values are `category`, `material`, `color`, `size`, `style`, `brand`, `budget`, `feature`, `use_case`, `other`.

Why it was needed

By consequence #3 in §3, this field is the agent's only means of learning anything. The baseline left it `null` permanently.

What changed — two implementations

`FixedPolicy` **(**`src/policy.py:40`**) — what ships.** Asks the broadest available question every turn, falling through a ladder (`feature`, `use_case`, `style`, …) as attributes are declined. Simple and, as measured below, effective.

`InfoGainPolicy` **(**`src/policy.py:72`**) — the more interesting design.** Chooses the question that would most reduce uncertainty about which candidate is correct.

The intuition, using colour. Suppose the shortlist holds 300 candidates: 150 black, 150 white. Asking "what colour?" eliminates about half whatever the answer is — a genuinely useful question. Now suppose 299 are black and 1 is white. The same question almost certainly returns "black" and eliminates almost nothing. **The value of a question is how evenly it splits what you're unsure about.** The formal name for that measure is **entropy**.

The policy scores each attribute as:

```
value(attribute) = coverage x gain_ratio x answerability
```

Each term earns its place, and two of them exist because the naive version failed:

- **coverage** — the fraction of candidates for which the attribute is even resolvable. This is

what stops the agent asking about budget: **78.9% of this catalog has** `price: null`, so a budget question is unanswerable most of the time.

- **gain_ratio** — entropy divided by its maximum possible value. *This was a bug fix.* Raw

entropy is biased toward attributes with many distinct values. Measured on a real session, brand scored **6.1 bits** against colour's **2.3** purely because the catalog holds thousands of distinct stores, so the agent opened every conversation by asking which brand the customer wanted. Normalising by `log2(number of distinct values)` removes the bias. This is a classic trap in information-gain methods and is worth knowing about generally.

- **answerability** — the probability a *shopper* can answer. Even after the gain-ratio fix, brand

still won: every product has a brand, so coverage was 100% and the split was near-perfect. But few people browsing for a belt can name the brand they want. This term encodes shopper behaviour, not catalog structure — and the customer's own `preference_tags` nudge it, which is the one place the user profile does real work.

Broad questions ("anything else that matters?") are scored on the same scale rather than special-cased: they can be answered as long as something remains undisclosed, and they return whichever fact the customer thinks is most important, so their value is the average gain across resolvable attributes scaled by how many facts a broad answer tends to surface.

Measured effect — and why the simpler one ships

	dev	holdout
`FixedPolicy`	**0.8738**	**0.8374**
`InfoGainPolicy`	0.8369	0.8141

`InfoGainPolicy` actually finds the target *slightly more often* on dev (hit rate 0.958 vs 0.950) — but ranks it considerably worse (MRR 0.703 vs 0.814). The cause is understood: specific questions surface fewer facts per turn than broad ones, and thinner evidence ranks worse even when it still retrieves.

The decision rule was fixed **before** the measurement: ship whatever wins on data that was not tuned against. `FixedPolicy` ships. `InfoGainPolicy` stays in the repo, fully documented and selectable:

```
Agent(catalog, AgentConfig(policy=InfoGainPolicy(agent.facets)))
```

Phrasing — pool-aware clarification wording (`src/phrasing.py`*)*

Separate from *what to ask* is *how to say it*. `FixedPolicy` keeps `ask_attribute="other"` because that is the score-optimal extraction (§3 consequence: the simulator returns two constraints of any type for `other` and can return zero for a specific attribute, which also retires that attribute), and the simulator never reads the English `message` field at all. So the sentence is free to be a real, guiding question while the machine-readable field stays put.

`clarify()` (`src/phrasing.py`) builds the message. From turn 2 onward — once the retrieval pool has been shaped by something the shopper actually said — it looks at the live reranked pool and, for each of `material / colour / style / size / use_case` that has not been asked or declined, measures how evenly the pool is split on it — the same `gain_ratio` (entropy ÷ maximum entropy) the `InfoGainPolicy` uses. Facets that are genuinely split (at least 25% of the pool resolves it, the top value holds no more than 90% of the mass, two or more values present) are collected, ordered by split quality, and the one at `turn_count % count` is voiced — so a session that stays on this path asks about a different facet each turn rather than repeating one. It names the top two or three values in one of ten complete sentence templates:

> "The materials I'm looking at come in leather and canvas — do you lean one way > on material?" > "So far the list covers black, gold and silver — any steer on colour?"

The turn-2 gate does **not** require a *productive* turn. Single-word disclosures ("leather", "black") never form a multi-word constraint span, so `productive_turns` can sit at 0 for a whole session that is in fact narrowing well — an earlier stricter gate left those sessions (e.g. `public_0198`) on the broad fallback for all ten turns. The per-facet split tests are the real guard against voicing a facet the pool has not split on. On the public set the grounded path now fires on 98% of turn-≥2 clarifications and in 199 of 200 sessions.

On turn 1, and whenever no facet qualifies, it falls back to a seven-way rotation of the broad question ("Anything else I should keep in mind?"); when a specific ladder rung is asked (`FixedPolicy.FALLBACK` after `other` is declined) it uses a three-way rotation of that attribute's own question ("Any must-have features?", "How should this fit?"). The whole path is wrapped so a phrasing bug degrades to a good question, never an empty turn. `brand` and `budget` and `category` are excluded from the *voiced* facets — brand has thousands of values, budget is null for 79% of the catalog, and the `category` facet's values are path fragments ("women", "novelty").

Lead-in and intent. The old code prefixed every sentence with the router's one fixed tone string ("To point you in the right direction: "). Now the prefix is drawn from a bank keyed on `route.name`: browsing sessions get soft framings ("To help me narrow things down, "), buying sessions get decisive ones ("To zero in on the right one, "), and half the bank is empty — so many turns carry no prefix at all, which is how people actually talk. On the turn the customer reverses course (`state.override_turn == turn_count`) the prefix is a distinct acknowledgement ("Okay, switching gears — ", "Got it, let's re-aim — ") and every turn after an override is treated as focused (buying prefixes), because a reversal is a decisive act.

Determinism. Each of the three banks (lead-in, grounded template, broad question) is indexed by `zlib.crc32` of the *opening line* plus the turn number (and, for the grounded template, the voiced attribute). Keying on the opening rather than the evaluator's random `session_id` means the wording is fixed per session and reproducible across runs, while still differing turn-to-turn and session-to-session. `natural_questions=False` bypasses all of this through `_legacy_tone` and reproduces the old fixed strings byte-for-byte.

This lives in S4 rather than S9 because it is the customer-facing half of the clarification decision, and it belongs *beside* `InfoGainPolicy` — it reuses the same pool-split measure, just to choose what to voice rather than what to ask.

It is deterministic and template-based on purpose: the facet vocabularies are small (§S1) so the space is enumerable, exactly like the rest of the pipeline. An LLM could later replace the grounded-question builder for fluency with this as its fallback; `ask_attribute` and the score do not move either way.

Measured effect

Exactly zero, by construction — the simulator ignores `message`. Verified in one process (`tools/sweep.py` rows `natural_off` / `natural_on`):

	dev	holdout	public (200)	adversarial (96)
`natural_questions=False`	0.9418	0.9136	0.9305	0.8020
`natural_questions=True`	0.9418	0.9136	0.9305	0.8020

Per-scenario components are identical on every split. The change buys demo / Presentation / Innovation realism (Pillar II's "structured, proactive clarification prompts"), not score. Default on; `AgentConfig(natural_questions=False)` restores the fixed strings byte-for-byte.

Ideas for this stage

- **Model expected *yield*, not just split quality.** This is the identified fix for

`InfoGainPolicy` and follows directly from the measurement above. It currently estimates *how much a known answer would narrow the field* but not *how many facts the answer will contain*. Broad questions win here precisely because they return two constraints instead of a fraction of one — and the policy cannot see that. Estimating yield per attribute from observed disclosure sizes, rather than the current fixed `broad_yield` constant, is the single change most likely to make the principled policy overtake the simple one.

- **Ask about the top candidates, not the whole pool.** Entropy is computed over 150 candidates

weighted by score. Concentrating on the top ~20 — the ones actually in contention — would ask questions that separate the leaders rather than the field.

- **Two-step lookahead.** The policy is greedy, picking the best single question. Some pairs of

questions are worth more together than either is alone.

- **Question phrasing from the candidates.** *Done* — `src/phrasing.py`, *see "Phrasing" above.*

The `message` now names the facet the live pool is most split on and its top values ("For the material, I'm seeing leather and canvas — do you have a preference?"), rotated so it does not repeat, while `ask_attribute` stays `other`. Deterministic, no model, score unchanged by construction. An LLM polish layer would slot in as `_grounded`'s replacement.

S5 — Retrieval

File: `src/retrieval.py`

What this stage does

Produces the shortlist of ~300 plausible candidates from 50,000, given everything the customer has said.

Why it was needed

The baseline ran a single query over the current message only. With state (§S3) the query can now be built from the whole conversation.

What changed

Two **routes** — independent searches — whose results are combined:

1. **Terms route** — all the customer's words, over the whole conversation. High recall.
2. **Focused route** — the same, but over post-override turns only. Runs only when the customer

has changed their mind, and gives the reversal a clean channel.

Combining with Reciprocal Rank Fusion (RRF). Two searches return two ranked lists with scores on incompatible scales — one might score 34.4, the other 18.7, and those numbers mean different things. RRF sidesteps calibration by **ignoring the scores and using only the positions**:

```
combined_score(product) = sum over routes of  weight / (60 + position)
```

A product at position 1 contributes `1/61`; at position 10, `1/70`. Appearing high in both lists beats appearing very high in one. Because it needs no calibration, adding a route later cannot destabilise the existing ones — and if a route returns nothing, it simply contributes nothing.

Measured effect

The terms route is **already excellent at its job**. Measured over 80 sampled sessions with all four constraints disclosed:

- target present in the shortlist: **80 / 80**
- median position of the target: **1**

- target within the top 10: **69 / 80**

That result set the direction for everything after it. **Retrieval was not the bottleneck — ranking was.** 80/80 of the targets were being found; 11 of them were simply ordered badly. So effort moved to §S6, and a planned dense-vector retrieval route was cancelled as unnecessary.

Ideas for this stage

- **Tune** `weight_focused`**.** Set to `0.8` by judgement (`src/retrieval.py:36`) and never swept.

Like `PRE_OVERRIDE_WEIGHT`, it sits directly on the weakest scenario.

- **Weight words by recency.** Every word in the conversation currently counts equally, whether

said on turn 1 or turn 5. Later disclosures are more specific and arguably deserve more weight — a cheap experiment on top of the existing `Utterance.weight` machinery.

- **A category route.** The opening message names a coarse category ("Jewelry Necklaces") that

currently contributes only as loose words. A dedicated route restricted to that branch of the category tree would suppress cross-category noise. (See also §S6, where the same idea may work better as a ranking signal.)

- **Dense retrieval, if recall ever becomes the bottleneck.** A local sentence-embedding route

would catch paraphrases that share no words. It was cut because recall is 80/80, and it would compromise the no-network guarantee — but if the private sessions paraphrase heavily, recall could fall and this becomes relevant again.

S6 — Reranking

File: `src/rerank.py`

> **The second-largest win in the project**, and the clearest illustration of the > retrieval-versus-ranking distinction from §2.

What this stage does

Takes the ~300 candidates and reorders them so the best is first.

Why it was needed

From §S5: the target was already in the shortlist 80/80 times at median position 1, but only reached the top 10 in 69 of 80. The gap was pure ordering. And ordering is worth a lot — MRR carries 30% of the score, and by consequence #1 in §3 you only ever get one chance at it.

What changed

The insight. The constraints the customer discloses are not paraphrases. They are **copied verbatim from the target product's own metadata** — `intent_card()` (`evaluator/local_evaluator.py:52`) builds them directly out of the product's `features` and `details` fields. So when a customer says "stainless steel band", those exact words are sitting in the target product's text.

That makes an unusually strong signal available: check whether each candidate's text *literally contains* the phrases the customer used.

```
customer said:  "stainless steel band", "day date indicator"

candidate A text: "...analog watch, stainless steel band, day date indicator..."  -> 2 matches
candidate B text: "...leather strap chronograph..."                               -> 0 matches
```

The final score for each candidate combines three signals:

```
score = span_coverage                  (dominant - verbatim phrase matches)
      + normalised_retrieval_score     (BM25's opinion, scaled to 0..1)
      + 0.4 x popularity               (the tie-break - raised from 0.02, see below)
```

Longer phrases count slightly more (`length_bonus`), because a five-word match is rarer than a two-word one.

Popularity was deliberately near-zero — and then measurement moved it. For most of this project `average_rating` and `rating_number` were held at weight `0.02`: the target is *one specific person's purchase*, not a bestseller, so a strong popularity prior would drag the ranking toward famous products. That reasoning is right in general and was wrong about the margin: change 12 dissected every session where the target sat at rank 2-10 behind an impostor and found all lexical signals *exactly tied*, the retrieval score picking the impostor 33/33 (BM25 rates the same matched words higher in a thinner listing), and popularity picking the target 31/33 — because a product someone really bought tends to be a reviewed, documented product. At 0.02 the signal that was right 94% of the time was drowned 50:1 by one that was wrong 100% of the time in exactly the regime that held all the remaining headroom. The weight is now `0.4` — mid-plateau, every split up, measured in `docs/team/rerank_signals.md` §10. The general lesson survives in refined form: popularity is a bad *primary* signal here (the coordinate-ascent argmax pushed it to 0.8 and regressed the adversarial set), but the *tie-break* is worth far more than a token weight.

Why this lives in ranking and not retrieval. Verbatim matching was *first* built as a third retrieval route, using FTS5 phrase queries. It scored **0.6859 against the floor's 0.7799 — a large loss.** Investigation showed why: as a search route it found the target in only **47 of 80** sessions, against the terms route's 80/80. Fused into retrieval, it injected 33 sessions' worth of noise ahead of good results.

The same evidence, applied to a shortlist the terms route had already filled correctly, is pure gain. **Precision-oriented signals belong in ranking; recall-oriented ones belong in retrieval.** This is the concrete example promised in §2.

**Later addition — matching the category *tail*.* *The opening message names the target's coarse category, and the evaluator builds that from the two most specific levels of the target's category path (*`coarse_category`*, *`evaluator/local_evaluator.py:126`*): a target filed under* `Novelty > Women` *produces* "I'm looking for Novelty Women"*.

Rewarding candidates that share *ancestors* with the opening cannot use this. Consider two candidates when the customer says "Novelty Women":

```
target      ... > Novelty > Women                            tail = "Novelty Women"    named
candidate   ... > Novelty > Women > Tops & Tees > T-Shirts   tail = "Tops & Tees T-Shirts"  not named
```

The candidate shares *every* ancestor the target has, so ancestor overlap scores it just as highly — yet its own two most specific levels were never mentioned. Scoring the tail separates them: award a point for each of the candidate's two deepest category levels whose words all appear in the opening. Matching is by token containment rather than by parsing the opening's sentence pattern, so a paraphrased private-set opening still works.

In the last remaining public-set miss (`public_0020`) this cut a **159-way tie** down to the few candidates on the right leaf. It is worth **+0.0058** on the public set and **+0.0053** on the adversarial set, entirely through better ordering — hit rate does not move on either set, which is exactly what a reranking signal should look like (`RerankConfig.tail_weight = 0.8`).

Measured effect

0.7799 → **0.8543** for verbatim span coverage — the single largest gain after dialog state. The signals added since (facet agreement, category agreement, and the category tail match) are recorded per change in `agent_changes.md`.

Ideas for this stage

- ~~**Facet agreement — the clearest unused signal in the repo.**~~ **Done.** `src/facets.py`

was extracting material, colour, size, brand and price band for every product while no ranking signal read any of it. It now feeds `RerankConfig.facet_weight` (PR #3).

- ~~**Category anchoring.**~~ **Done, twice.** The motivating failure was a target belt whose

customer had disclosed only `"buckle closure"` and `"100 leather"` — both generic — leaving the target at **rank 15** behind a dozen other leather belts. This is now covered by `category_weight` (PR #4) and, more sharply, by the category tail match described above. Together they closed the last public-set miss.

- ~~**Learn the signal weights.**~~ **Done — with a twist** (change 12). `tools/fit_weights.py`

fits all seven non-definitional weights by coordinate ascent directly on the technical score (Metzler & Croft 2007), dev split only, standard library, offline. The sealed holdout confirmed the fit's direction (+0.019), but the dev argmax regressed the adversarial set, so what shipped is the single change the fit and the near-miss anatomy both pointed at: `popularity_weight` 0.02 → 0.4. Public 0.9199 → 0.9305, every split up. The honest lesson: the fit finds the direction; the gates decide the magnitude.

- **A negative-evidence signal.** Nothing currently penalises a candidate that *contradicts* a

stated constraint. A customer who said "leather" and a candidate whose material is explicitly canvas should be pushed down, not merely left unrewarded.

- **What was already tried and rejected here:** weighting phrases by how rare they are within the

shortlist. It moved dev by `0.0002` and holdout not at all — a pool retrieved by those same words has little rarity spread left to exploit. The code was deleted rather than kept as a dead option; see §6.

S7 — Recommendation timing

File: `starter/agent.py` (`_shortlist`, `_confident`)

What this stage does

Decides whether to show a list this turn, or stay quiet and ask another question.

Why it was needed

This stage exists entirely because of consequence #1 in §3: **the session ends at the first hit, freezing the rank forever.** Showing an uncertain list early risks catching the target at position 9 and locking in `RR = 0.11`, when waiting two more turns might have ranked it first.

The marginal-value table in §3 quantifies the trade: a rank improvement is worth ~13x the turn it costs. So the agent should be patient.

What changed

Hold until turn 3. No recommendations on turns 1-2, when the customer has disclosed little. Measured on holdout:

First recommendation on	Score
turn 2	0.8088
turn 3	**0.8349**
turn 4	0.8196

Turn 3 wins on both splits. Turn 2 is too eager; turn 4 wastes turns after the evidence has arrived.

Confidence gating. A fixed turn number is crude — sometimes the answer is obvious on turn 2. `_confident()` allows early recommendation when the top candidate clearly leads:

```
(best_score - runner_up_score) / best_score  >=  0.20
```

Margin	holdout
0.0 (disabled)	0.8349
0.15	0.8391
0.30	0.8389
0.50	0.8371

Every enabled value beats disabled, on both splits, and **MRR is unchanged while MTTC improves** — exactly the intended mechanism. The agent gets faster without ranking worse.

The default is `0.20`, not `0.15`. `0.15` scored highest, but the curve from `0.15` to `0.50` is flat, which means the differences between them are noise. Picking a split's argmax is how you buy noise. Mid-plateau is the defensible

choice — the same reasoning applies to `broad_yield` in `src/policy.py`, whose dev results (`0.8530` at 5.0, `0.8651` at 8.0, `0.8634` at 12.0) are one plateau.

Narrow the first slate. `list_size_ramp` says how many candidates each turn reveals; the last entry applies to every later turn. It shipped flat at `(10,)` for a long time, and is now `(4, 10)` — four candidates on turn 3, ten from turn 4.

Showing *fewer* products scores better, which sounds backwards until you line up three facts about the evaluator. The session ends the instant the target appears in a shown list. The rank it held at that instant becomes the reciprocal rank, permanently. And a list that misses costs nothing but a turn.

So every list is a bet. Reveal ten on turn 3 and you maximise the chance of converting *now* — at whatever rank the target happens to hold. If it is sitting at position 7, you have just banked `RR = 1/7 = 0.14` and the session is over. Reveal four, and that same target stays hidden; you spend a turn, the customer discloses another constraint, the reranker re-scores with it, and the target typically surfaces near the top. Now you bank `1/1` or `1/2`.

The elimination scan is what makes this safe. Products already shown are excluded from later lists, so the six held back on turn 3 are simply the top of the survivor list on turn 4 — nothing is skipped, the same walk is just taken in smaller steps.

First slate	dev	holdout	generated	hard
10 (flat)	0.9233	0.9048	0.9181	0.7944
3	0.9254	**0.9146**	**0.9212**	0.7968
4	0.9268	0.9096	0.9197	0.7981
5	**0.9295**	0.9100	0.9210	**0.8001**

Three, four and five all beat the flat ramp on all four sets — that is a plateau, so `4` ships as its midpoint rather than `5`, which happens to top two columns. The decision rule was fixed before `4` was measured, which is the point: choosing the winner after seeing the table is how you buy noise, exactly as with the `0.20` margin above.

Narrowing a *second* turn is not more of the same good thing. `(5, 5, 10)` scores `0.9272 / 0.9044 / 0.9187 / 0.7934`, dropping holdout and hard below the flat floor. Each session holds four constraints and the customer discloses at most two per turn, so by turn 4-5 no further evidence is coming and waiting longer spends turns without buying rank.

Measured effect

Confidence gating: `0.8543` → `0.8592`, with MTTC improving from 3.58 to 3.41.

Narrow first slate: public `0.9159` → `0.9199`, adversarial `0.7944` → `0.7981`, dev `0.9233` → `0.9268`, holdout `0.9048` → `0.9096`. The whole gain is MRR bought with MTTC at the ~13x odds §3 prices — on public, MRR `0.8513` → `0.8690` (×0.30 = +0.0053) against MTTC `2.975` → `3.040` (efficiency –0.0065, ×0.20 = –0.0013), netting the +0.0040 observed. Hit@10 does not move on public (1.000) or hard (0.885): this buys rank, not coverage. One cost worth naming — on the 200-session generated set Hit@10 slips `0.995` → `0.990`, a single session that now runs out of turns.

Ideas for this stage

- **A calibrated stopping rule.** The margin test is a heuristic on raw score gaps. Converting the

reranker's output into an actual probability that the top candidate is correct — fitted on the public sessions — would let the agent stop on a meaningful threshold ("80% confident") instead of an arbitrary ratio.

- **Spend the remaining turns.** The agent stops at the first hit, but it does not know that a hit

occurred. On sessions heading for a miss it still shows the same list every turn from turn 3 onward. Detecting stagnation — the shortlist stopped changing, the customer stopped disclosing — and switching to a different strategy would attack the 6% of sessions that currently miss entirely.

- **~~Diversify a low-confidence list.~~ Tested, and it does not work.** This entry used to argue

that spreading the list across distinct categories or brands would raise the chance of catching the target, on the grounds that a miss→hit is worth 2.6x a rank improvement. **That premise is stale:** it was written when Hit@10 was `0.940`. Hit@10 on the public set is now `1.000`, so there are no misses left to convert and the trade has nothing to buy. An MMR diversity term was implemented and measured anyway: across 260 public slates it brought the target *into* a

slate zero times and pushed it *out* 21 times, and Hit@10 moved on no dataset at any setting. The reason is structural — the customer's disclosed constraints are copied verbatim from the target's own metadata, so the crowded top of the list is a cluster formed *around the target*, which sits at its centre rather than being an outlier crowded out of it. `docs/team/ideas.md` Idea 3 records the full measurement.

- **~~Vary list length with confidence.~~ Done — see "Narrow the first slate" above.** This entry

used to guess that "there is likely no benefit to showing fewer". The opposite is true, and for a reason the guess missed: showing fewer is not about precision, it is about *when you commit*. `list_size_ramp` now ships at `(4, 10)`, worth `+0.0040` on the public set. The remaining unexercised part of the idea is making the width depend on measured confidence rather than on the turn number — narrow while `_confident()` is false, ten once it is true.

S9 — Robustness and tests

Files: `starter/agent.py`, `tests/`

What this stage does

Ensures the agent never crashes, always emits a valid response, and that changes cannot silently break the score.

Why it was needed

Consequence #4 in §3: the evaluator swallows exceptions and turns them into missed sessions with no error shown. A crash on turn 1 of every session would produce a score of zero and an entirely clean-looking run.

What changed

`respond()` **cannot raise.** The whole path is wrapped; any exception degrades that turn to a valid empty response rather than losing the session. Each stage also degrades locally — a malformed search expression returns `[]`, an illegal attribute is coerced to `other`, a missing session is recreated rather than raising.

25 tests, in three groups:

File	Covers
`tests/test_contract.py`	Every response validates against `docs/agent_api_contract.json`; hostile input (empty, 5000 characters, SQL) never raises; sessions don't leak into each other; `top_k` respected
`tests/test_components.py`	Each stage in isolation — including a test asserting the gain-ratio fix, so the brand-cardinality bug cannot silently return
`tests/test_regression.py`	End-to-end score on the **holdout** split must stay above **0.78** — the committed floor

The regression test is the important one. The pipeline has many interacting knobs and it is easy to make a local improvement that costs score overall; this is what catches that.

```
python3 -m unittest discover -s tests -t .   # 25 tests, ~14s
```

Measured effect

No score gain. It is insurance — and it caught two real bugs during development, including a policy that kept re-asking a question the customer had already declined.

Ideas for this stage

- **Assert the fallbacks never fire.** Defensive layers can mask real bugs. Counting fallback

activations and asserting the count is zero on a clean public run would distinguish "robust" from "quietly broken".

- **Property-based testing.** Generating random conversations and asserting invariants (never

crashes, never recommends a non-catalog ID, never re-asks a dead attribute) would cover far more input shapes than 25 hand-written cases.

- **A performance regression test.** Latency is currently measured ad hoc. Pinning per-turn

latency in a test would catch an accidentally quadratic change before judging day.

---

6. What was tried and rejected

These are first-class results. They are the evidence that the shipped design was **chosen** rather than assumed, and each one is recorded in the relevant module's docstring so the next reader does not repeat the experiment.

Experiment	Result	Outcome
Verbatim matching as a retrieval route	0.6859 vs floor 0.7799	Removed from retrieval, moved to reranking — where it became the second-largest gain. Found the target in only 47/80 sessions as a search route.
Routing pool size and timing by intent	dev unchanged, holdout –0.002	Router scoped down to phrasing only.
Weighting phrases by rarity within the shortlist	+0.0002 dev, 0.0000 holdout	Deleted entirely, not left as an off-by-default flag.
`InfoGainPolicy` **as the default**	dev 0.8369 vs 0.8738; holdout 0.8141 vs 0.8374	Kept in the repo and documented, but not the default.
Dense-vector retrieval	not built	Cancelled after measuring retrieval recall at 80/80 — the bottleneck was ranking, and it would have broken the no-network guarantee.
Turn-1 exclusion from facet extraction	public 0.9159 → 0.9150, holdout 0.9048 → 0.9035	Rejected. Removes every false conflict against the target (8 public, 5 hard) and still loses — the opening's category words are a real constraint, and the impostors they demote outweigh the targets they cost.
Multi-valued facet extraction	agreement –0.0006 public / –0.0023 hard; conflict-only 0.0000	Rejected. The agreement half dilutes the signal by matching more candidates; the conflict half is exactly neutral and was not shipped, because a code path no measurement justifies does not earn its place.
Explicit constraint ledger (S3)	not built	Cancelled after specifying and measuring all six operations — see below.
Neural cross-encoder reranking (S6b)	dev 0.9268 → 0.9211, hard 0.7981 → 0.7944	Built, measured, removed. Loses on every split and every setting; the optimum semantic weight is zero. Code preserved on branch `semantic-rerank`.

The constraint ledger, and why measuring a design beats arguing about it. A typed `Constraint` ledger (slot, value, turn, polarity, status) with CARRY / UPDATE / ADD / DELETE / DONTCARE / NEGATE operations was proposed as a replacement for replaying raw utterances. Rather than build it and see, each operation was checked against what the evaluator actually does. Four of the six could not pay: `CARRY`/`ADD` is what `full_text()` already does, `DONTCARE` is already implemented as `Utterance.declined` plus `dead_attributes`, and `DELETE`/`NEGATE` cannot fire because `customer_reply()` only ever *adds* constraints.

The sixth, `UPDATE`, is the interesting one, and it failed for a reason worth writing down. `behavior_for()` draws both the discarded and the new preference from the **same target product's** intent card. Across all 46 override sessions in the public and hard sets, **not one replaces an exclusive facet value with a different one**: 25 of 30 public overrides are cross-slot (`"Buckle closure"` → `"leather"` — a material is added, the buckle is still true), 4 are `feature` → `feature` (`"Stainless Steel Band"` → `"Water Resistant"`, both true), and the single `material` → `material` case repeats the same value. What reads as "ignore my earlier preference" is an *emphasis shift*, not a retraction. There is nothing to supersede.

Chasing this also corrected a wrong explanation in the shipped code. `src/rerank.py` judged conflicts against `focused_text()` and attributed that to stale post-override values punishing the target. Measured: single-value extraction over full history picks a contradicted value in **0 of 30** sessions. The guard's real function is dropping turn 1,

whose `coarse_category()` framing ("I'm looking for Pants Casual") extracts as a `style` constraint. Two variants — exclude turn 1 keeping `focused_text()`, and exclude turn 1 using full history — scored **bit-identically on all four splits**, which is the proof. The guard stays; only its comment changed. Full measurements in `docs/team/rerank_signals.md` §6-§8.

The cross-encoder, and what "the model is wrong" actually looks like. Every rerank signal in this agent is lexical — it counts words the customer said that also appear in a product's text. The obvious next move is a *semantic* model that scores how well a product matches a request even when the words differ. `src/semantic.py` does exactly that, with a small neural cross-encoder (`cross-encoder/ms-marco-MiniLM-L6-v2`, 22.7M parameters, Apache-2.0) that reads a (request, product) pair together and emits one relevance score.

Before building it, we measured the ceiling. An **oracle** reranker — one that cheats by forcing the true target to position 1 whenever it is anywhere in the candidate pool — scores 0.9631 on the public set against the shipped 0.9199, and 0.8823 on the adversarial set against 0.7981. So *any* reranking improvement, neural or not, is playing for +0.043 and +0.084. That is worth having, and it is also the whole prize: 70% of public sessions already rank the target first.

The result was negative on every split and every setting:

variant	dev (120)	hard (96)
off (shipped)	**0.9268**	**0.7981**
cross-encoder, weight 0.7	0.9211	0.7944
cross-encoder, weight 0.3	0.9249	0.7959
cross-encoder, depth 20	0.9236	0.7940

Read the weight column downward: 0.7 → 0.3 → 0 recovers the baseline monotonically. **When the best setting for a signal's weight is zero, the signal carries no usable information.** There is no threshold to tune toward.

The mechanism is one number. On the 162 turns where the gate fired and the target was in the rescored group, the model moved it **up 46 times and down 74** — mean position 7.63 → 8.77. It is not slightly miscalibrated; it is pointing the wrong way more often than the right way. The likely reason is a mismatch between what the model was trained on and what it was asked to do here. Its training data pairs a natural-language question ("how tall is the Eiffel tower") with a prose passage. Here the "question" is simulator boilerplate — *"For that, what matters is: full grain leather; buckle closure"* — and the "passage" is a token-joined blob of title, features and description. On top of that, the job it was given is the hardest discrimination in the pool: separating near-identical products that share every stated attribute, which is precisely where the lexical signals had already saturated.

Cost, for the record: mean turn latency 30.7 ms → 389.8 ms, p95 73.7 ms → 1347.8 ms, worst turn 1.48 s, with the gate firing on 28% of rerank calls. `docs/competition_specification.md` notes that timeouts may count as a miss, so even a *positive* result at this latency would have needed care.

The stage was removed from the working branch and preserved on `semantic-rerank`: it never ran on the scored path — disabled by default, and a silent no-op without its gitignored weights — so carrying it would have meant carrying a dependency manifest and a download tool for code that never executed. The measurement is the deliverable, and it survives in full in `docs/team/rerank_signals.md` §9, along with the oracle ceiling it established.

A bug worth learning from. The first version of `InfoGainPolicy` opened every conversation by asking the customer's preferred *brand*. The cause was using raw entropy: brand has thousands of distinct values in the catalog, so it scored 6.1 bits against colour's 2.3 and always won — even though it is close to useless as a shopping question. This is a well-known failure of information-gain methods, and the fix is standard: divide by the maximum possible entropy to get a **gain ratio**. `tests/test_components.py` now asserts brand scores below colour, so the bug cannot return.

---

7. Results

How the score was built up

Stage of work	Hit@10	MRR	MTTC	Score
Supplied baseline	0.125	0.068	9.81	**0.1067**

Stage of work	Hit@10	MRR	MTTC	Score
+ dialog state and asking questions (§S3, §S4)	0.870	0.694	4.17	**0.7799**
+ verbatim span reranking (§S6)	0.940	0.786	3.58	**0.8543**
+ confidence gating and policy fixes (§S7)	0.940	0.791	3.41	**0.8592**

(An early prototype of the state-and-questions step measured 0.7811 before the code was reorganised into modules; 0.7799 is the same policy re-measured in the final structure. The small difference comes from boilerplate word handling added in §S1.)

Dev versus holdout

Split	Sessions	Hit@10	MRR	MTTC	Efficiency	Score
Public (all)	200	0.940	0.791	3.41	0.760	**0.8592**
Dev (tuned on)	120	0.950	0.814	3.27	0.772	0.8738
Holdout (untouched)	80	0.925	0.756	3.60	0.740	0.8374

Dev is higher than holdout, which is expected and healthy — dev is where the tuning happened. The gap of `0.036` is within what 80 sessions can produce by chance. The holdout number is the honest one to quote, and the one the regression test guards.

By scenario

Scenario	Sessions	Hit@10	MRR	MTTC
boundary	10	1.000	0.846	4.00
browsing	80	0.963	0.811	3.08
buying	80	0.938	0.794	3.21
intent_override	**30**	**0.867**	**0.713**	**4.60**

Intent override is the weakest scenario and the clearest target for further work. Part of the gap is structural — those sessions cannot convert before the override arrives on turn 3 or 4 (consequence #5 in §3), which puts a floor under MTTC. The rest is genuine, and §S3's ideas are aimed at it.

Operational

Model / API	**None.** No LLM, no network, no credentials.
Dependencies	Python standard library only
Cost	$0.00
Tokens	0 prompt, 0 completion
Index build	4.13s, once at startup
Per-turn latency	16.6ms mean, 19.8ms p95, 21.0ms max
Peak memory	~282 MB
Full 200-session evaluation	~28s

The agent behaves identically with the network disabled, because there is no online path to fall back from. This was deliberate: the submission rules reserve the right to score under network restrictions, and an agent scoring zero in that environment is worth less than one scoring 0.8592 everywhere.

---

8. Why upstream Amazon data was ruled out

The catalog derives from the public Amazon Reviews 2023 dataset, which contains far more than the 50,000 frozen products. Using it was considered and **deliberately rejected**. The reasoning, in order of strength:

1. It cannot add information to the channel that matters. This is decisive. Every constraint the simulated customer can ever utter is built by `intent_card()` (`evaluator/local_evaluator.py:52`) from `features`, `details`, `price`, and `searchable_text()` over `title` / `features` / `details` / `description` / `categories` / `store` (`evaluator/local_evaluator.py:22`). **All of those fields are already complete in** `data/catalog.jsonl`**.** The process generating the dialogue is closed over the frozen catalog, so upstream data cannot tell us more about *what the customer will say* — and interpreting what they say is the entire task.

2. It does not address the measured bottleneck. Retrieval already recalls the target 80/80 at median rank 1 (§S5). The losses are in ranking near-identical candidates. More product text does not break those ties.

3. It could actively hurt. The traced belt failure was *too many candidates matching generic phrases*. Richer descriptions give non-target products more surface on which to match. And we have direct evidence of this class of signal backfiring: the popularity prior had to be held to weight `0.02` (§S6) because the target is one specific purchase, not a bestseller.

4. Rule risk with a bad asymmetry. `docs/competition_specification.md:13` places "private-label reconstruction" and "identifiers outside the frozen catalog" out of scope, and the organizer deliberately stripped review text, user IDs, timestamps and purchase history for anonymisation. Re-attaching that data by `parent_asin` works against stated intent even where it is not literally banned — risking a 0.86 result for a marginal gain.

5. Disproportionate cost. Tens of gigabytes of download plus a distillation script, producing an asset that must then be committed to preserve the no-network guarantee. It breaks the one-command reproduction story for very little.

The honest hole in this argument: `materialize_hidden_fields()` (`evaluator/local_evaluator.py:204`) short-circuits when a session already carries an `intent_card`, so the private sessions *could* in principle ship cards built from richer data than the frozen catalog. That seems unlikely given the organizer froze the catalog as the shared artifact, but it cannot be verified from here.

The legitimate half, kept. The one genuinely useful upstream idea — mining real attribute vocabularies instead of hand-writing keyword lists — can be done against the frozen 50,000-product catalog itself. Same benefit, no download, no rule risk. It appears in §10.

---

9. Files at a glance

File	Stage	What to look at
`starter/agent.py`	adapter, S7, S9	`respond()` for the whole turn flow; `_confident()` for timing
`src/index.py`	S1	`CatalogIndex._build()`; `DEFAULT_WEIGHTS` at line 25
`src/text.py`	S1	`constraint_spans()` — how customer phrases are normalised
`src/router.py`	S2	`classify()`; the docstring records the scope-reduction measurement
`src/state.py`	S3	`DialogState.observe()`; `apply_override()` for the deviation
`src/policy.py`	S4	`FixedPolicy` (line 40, ships); `InfoGainPolicy` (line 72)
`src/retrieval.py`	S5	`retrieve()` and `_rrf()`
`src/rerank.py`	S6	`rerank()` — the span-coverage loop
`src/facets.py`	shared	`extract()`; currently unused by ranking (see §S6 ideas)
`tools/sweep.py`	S0	`build_configs()` — add a row to test a change
`tests/`	S9	`test_regression.py` guards the 0.78 floor
`ARCHITECTURE.md`	—	Data-flow diagram and stage boundaries
`README.md`	—	Results and decision rationale

Never modify `evaluator/`, `data/`, and the five frozen files at the root of `docs/` (`competition_specification.md`, `agent_api_contract.json`, `evaluation_config.json`, `baseline_results.json`, `submission_rules.md`) — they are organizer-owned and scoring is invalid if they change.

---

10. Cross-cutting and orchestration ideas

These are not tied to one stage — they concern how the stages work *together*, or they touch several at once.

Orchestration

Close the loop between asking and ranking. Right now the pipeline is a one-way pass: state → retrieve → rerank → ask. The clarification policy reads the candidate pool, but nothing feeds *ranking confidence* back into *what to ask*. A genuinely adaptive agent would notice "my top two candidates differ only in colour" and ask about colour specifically — a question chosen to break the current tie rather than to split the pool in general. This is the most interesting unbuilt idea in the project, it needs no new data, and it connects S4, S6 and S7 into an actual feedback loop rather than a pipeline.

Adaptive orchestration by session state. Every session runs the identical sequence of stages. A session where the customer is disclosing freely and a session where they keep saying "no preference" call for different behaviour — the first should keep asking, the second should stop asking and start showing lists. The signals to detect this already exist (`productive_turns`, `dead_attributes` in `src/state.py`); nothing consumes them for orchestration.

A per-turn budget. All stages run every turn regardless of whether they can contribute. The focused retrieval route only matters after an override; reranking only matters once spans exist. Skipping stages that cannot help would cut latency, and more importantly would make the agent's reasoning legible: "I skipped reranking because nothing has been disclosed yet."

Explain the recommendation. The reranker knows exactly why each candidate scored well — which phrases matched. Surfacing that in the customer-facing `message` ("these all have a stainless steel band and a date window") costs nothing, is listed under Innovation Directions in `docs/competition_specification.md:87`, and makes the demo far more compelling than a bare list.

Cross-cutting

Learn the weights instead of setting them. At least eight constants across the pipeline were set by hand or inherited: `DEFAULT_WEIGHTS` (S1), `weight_focused` (S5), `PRE_OVERRIDE_WEIGHT` (S3), `span_weight` / `retrieval_weight` / `popularity_weight` / `length_bonus` (S6), `confidence_margin` (S7). There are 176 known-correct public sessions to learn from. A single offline fitting pass — plain logistic regression, entirely within the standard-library and no-network constraints — would likely beat hand-tuning across the board and would be more defensible than a series of individual sweeps. **This is the highest-expected-value cross-cutting change.**

Mine facet vocabularies from the frozen catalog. `src/facets.py` uses hand-written keyword lists for material, colour, style, size and use case. Harvesting these from the 50,000 products themselves would be broader, less arbitrary, and would improve both the clarification policy (S4) and the proposed facet-agreement ranking signal (S6). This is the legitimate half of the rejected upstream idea, with none of its downsides.

An error-analysis tool. The remaining 6% of sessions that miss have never been examined as a group. A script that dumps every missed session with its disclosed constraints, the target's actual rank, and what outranked it would likely reveal one or two systematic causes worth more than any parameter sweep. The belt case in §S6 was found this way, by hand, and immediately suggested category anchoring.

Make robustness measurable, not assumed. The no-network guarantee, the never-raises guarantee and the latency budget are all currently claims backed by design rather than by continuous verification. Tests that actively enforce them (§S9 ideas) would turn three assertions in this document into three facts.

Glossary

Term	One-line meaning
BM25	Text-search scoring formula. Rare words count more; matches in short important fields count more.

Term	One-line meaning
Constraint span	A short phrase the customer said, normalised so it can be matched against product text.
Dev / holdout	120 sessions used for tuning / 80 kept untouched to check the tuning generalises.
Entropy	A measure of how evenly a set is split. High entropy = very uncertain = a useful question to ask.
Facet	A structured product attribute with limited values — material, colour, price band.
FTS5	SQLite's built-in full-text search engine. Requires no installation.
Gain ratio	Entropy divided by its maximum, to stop attributes with many values from always winning.
Hit Rate@10	Fraction of sessions where the target appeared in a shown top-10 list.
IDF	Inverse document frequency — the "rare words matter more" weighting inside BM25.
MRR	Mean Reciprocal Rank — average of `1/position`. Rewards ranking first, not merely appearing.
MTTC	Mean Turns To Conversion — average turn on which the hit happened; a miss counts as 11.
Overfitting	Tuning until you fit the noise in your test data, so results don't transfer to new data.
`parent_asin`	The product ID. The only thing scored, and only on exact match.
Provenance	Metadata about *when and under what circumstances* a piece of information arrived.
Reciprocal rank	`1 / position`. Rank 1 → 1.0, rank 10 → 0.1, miss → 0.
Reranking	Reordering an existing shortlist to put the best candidate first.
Retrieval	Finding a shortlist of candidates from the full catalog.
RRF	Reciprocal Rank Fusion — combining ranked lists using positions rather than scores.
Stopword	A common word removed before searching because it carries no signal.
Target	The hidden product the agent must find.
Top-k	The first *k* items of a ranked list; here *k* = 10.